WAT4

Web Application Testing

E2E Testing with Playwright

J. Karder

Playwright

playwright noun

play·wright ˈplā-ˌrīt

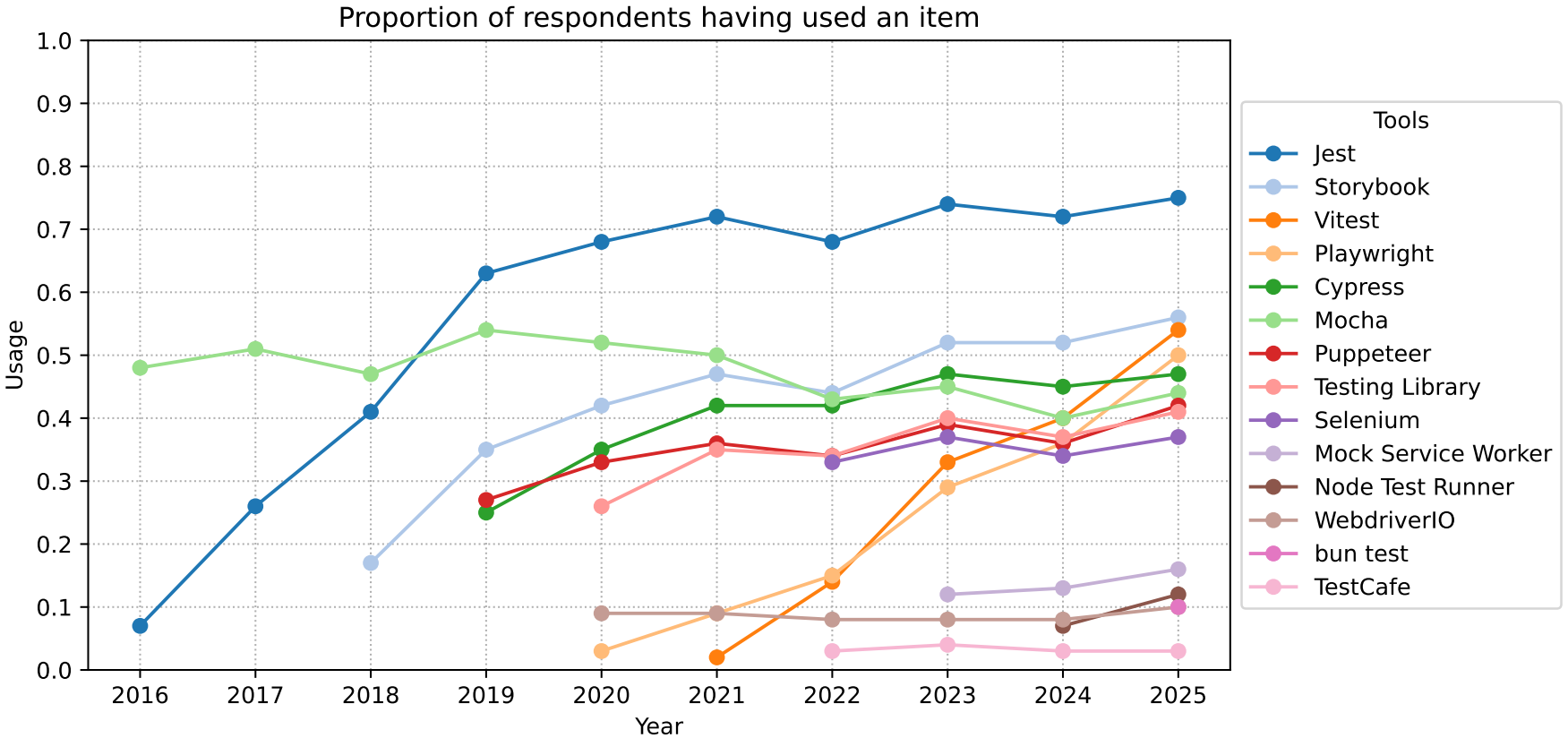
Synonyms of *playwright* >

: a person who writes plays

About (1)

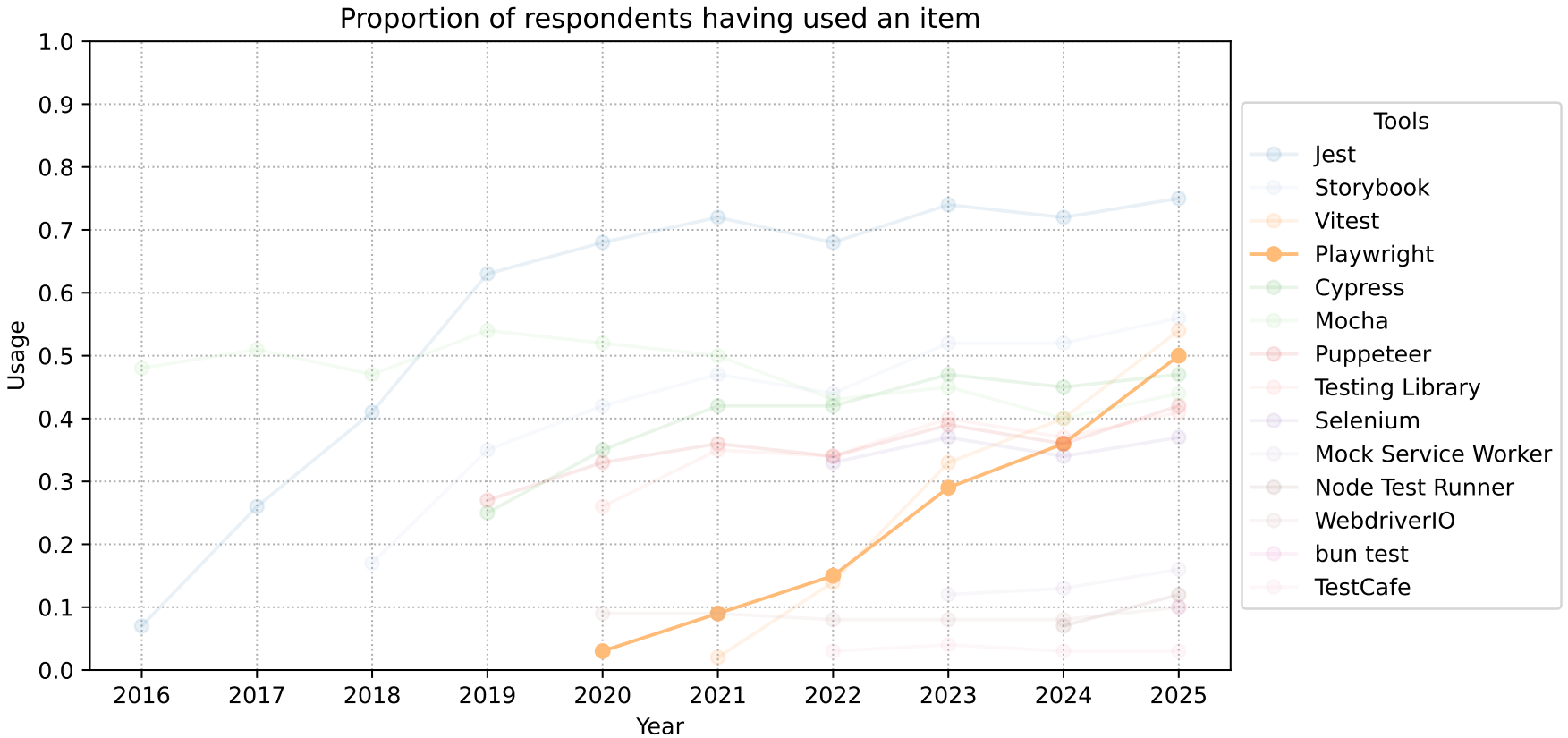
- End-to-end (E2E) test framework for web applications
  - Developed by Microsoft and launched on 31 January 2020
  - API for automating browser tasks
    - Supports Chromium, Firefox and WebKit browsers
    - Enables reliable E2E testing in both headless and non-headless modes
    - Works locally and in continuous integration pipelines
  - Bundles test runner, assertions, isolation, parallelization and rich tooling
  - Main API originally written in Node.js
    - Compatible with Python, Java and .NET (C#)
  - Supports modern web features like network interception and multiple browser contexts
  - Includes automatic waiting to minimize test flakiness
- GitHub stars: +84k
- GitHub forks: +5.3k

About (2)



Source: https://2025.stateofjs.com/en-US/libraries/testing/

About (2)



Source: https://2025.stateofjs.com/en-US/libraries/testing/

Getting Started

- Initialize a new project:

```
npm init playwright@latest
```

  - Also adds Playwright to an existing project
  - Downloads required browser binaries and creates the scaffold below:

```
playwright.config.js          # Test configuration
package.json
package-lock.json             # Or yarn.lock / pnpm-lock.yaml
tests/
  example.spec.js             # Minimal example test
```

Getting Started

- Update Playwright:

```
npm install -D @playwright/test@latest
npx playwright install --with-deps
```

  - Also downloads new browser binaries and their dependencies

Getting Started

- Run tests:

```
npx playwright test
```

  - Runs tests in headless mode and in parallel across Chromium, Firefox and WebKit (configurable in `playwright.config.ts`)
  - Shows output and opens test report if tests fail

- Show test reports:

```
npx playwright show-report
```

@playwright/test

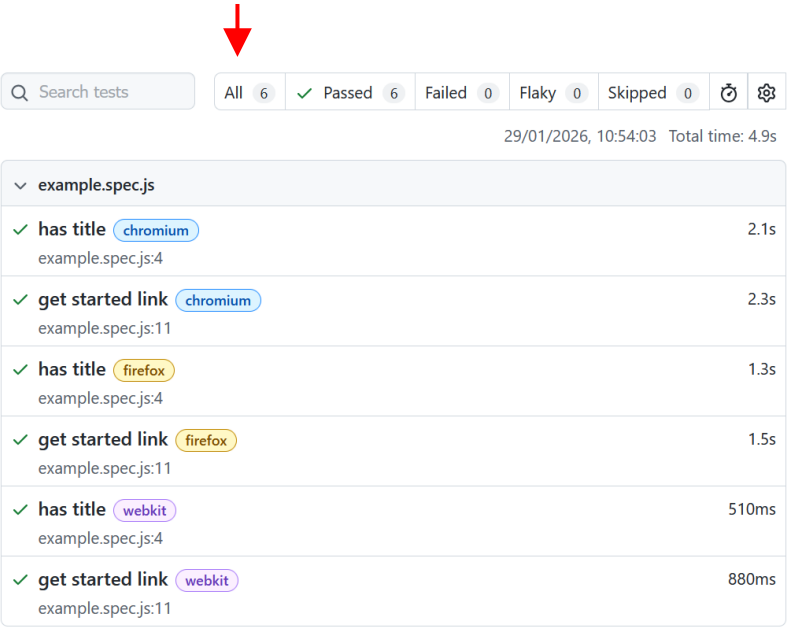
- Test runner developed and maintained by the Playwright team
- Built on top of the Playwright API with Jest-like assertions
- Tests are simple
  - Perform actions
  - Assert the state against expectations

  } AAA pattern
- Playwright automatically waits for actionability checks to pass before performing each action
  - No need to add manual waits or deal with race conditions
  - Assertions are designed to describe expectations that will eventually be met
  - Eliminates flaky timeouts and racy checks
  - More on this later ...

First Test

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Expect a title "to contain" a substring.
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Click the get started link.
  await page.getByRole('link', { name: 'Get started' }).click();

  // Expects page to have a heading with the name of Installation.
  await expect(page.getByRole('heading', { name: 'Installation' }))
    .toBeVisible();
});
```

```
$ npx playwright test

Running 6 tests using 6 workers
  6 passed (4.9s)

To open last HTML report run:

  npx playwright show-report
```



Actions

- Playwright offers a variety of actions (to AAA on a page)
  - Navigate to a specific page
  - Locate elements via Locators
  - Interact with found elements
  - Assert states against expectations

Anatomy of a Playwright Test

- Most tests start by navigating to a URL:

```
await page.goto('https://playwright.dev/');
```

- After that, the test interacts with page elements:
  - Elements are found using Locators

```
// Create a locator.
const getStarted = page.getByRole('link', { name: 'Get started' });

// Click it.
await getStarted.click();
```

Anatomy of a Playwright Test

- Playwright includes test assertions in the form of an expect function
  - To make an assertion, call `expect<T>(actual: T, ...)` and choose a matcher that reflects the expectation
  - Playwright includes async matchers that wait until the expected condition is met
    - Makes tests non-flaky and resilient
    - The following waits until the page gets the title containing the string "Playwright":
      - Note that one can use regular expressions to match strings

```
await expect(page).toHaveTitle(/Playwright/);
```

    - Such expectations will timeout after a specified duration

Locators

- Central piece of Playwright’s auto-waiting and retry-ability
- Represent a way to find elements on the page at any moment
- Recommended built-in locators:
  - `page.getByRole()` to locate by explicit and implicit accessibility attributes
  - `page.getByText()` to locate by text content
  - `page.getByLabel()` to locate a form control by its associated label’s text
  - `page.getByPlaceholder()` to locate an input by placeholder
  - `page.getByAltText()` to locate an element, usually image, by its text alternative
  - `page.getByTitle()` to locate an element by its title attribute
  - `page.getByTestId()` to locate an element based on its data-testid attribute
    - other attributes can be configured

Locators

- Locate the element by its role of button with name "Sign in":
  - Note that this retrieves an up-to-date DOM element from the page

```
const locator = page.getByRole('button', { name: 'Sign in' });
```

- An up-to-date DOM element will be located once prior to every action
  - Ensures that if the DOM changes in between the calls, the new element corresponding to the locator will be used

```
const locator = page.getByRole('button', { name: 'Sign in' });

await locator.hover();
await locator.click();
```

Locators

- Can also be chained:
  - Note that this allows to iteratively narrow down a locator

```
const locator = page
    .frameLocator('#my-frame')
    .getByRole('button', { name: 'Sign in' });

await locator.click();
```

Locators

- When to use which locators?
  - Prioritize **role locators** (`page.getByRole()`) to locate elements, as it is the closest way to how users and assistive technology perceive the page
  - Use **label locators** (`page.getByLabel()`) when locating form fields
  - Use **placeholder locators** (`page.getByPlaceholder()`) when locating form elements that do not have labels but do have placeholder texts
  - Use **text locators** (`page.getByText()`) to find non interactive elements like div, span, p, etc; for interactive elements like button, a, input, etc. use role locators
  - Use **alt text locators** (`page.getByAltText()`) when your element supports alt text such as img and area elements
  - Use **title locators** (`page.getByTitle()`) when your element has the title attribute
  - Use **test id locators** (`page.getByTestId()`) when you choose to use the test id methodology or when you can’t locate by role or text
  - Do not use **CSS locators** and **XPath locators** (`page.locator()`) unless there is no other way; these locators are not recommended as the DOM can often change leading to non resilient tests; instead, try to come up with a locator that is close to how the user perceives the page such as role locators or define an explicit testing contract using test ids

Actions – Examples

- `click()` an element
- `check()` and `uncheck()` an input checkbox
- `hover()` over an element
- `fill()` a form field, i.e., input text
- `focus()` an element
- `press()` a single key
- `selectOption()` to select an option of a dropdown

Auto-Retrying Assertions – Async Matchers

- Async matchers can be awaited until the expected condition is met, or a timeout occurs
- Examples:
  - `toBeChecked()` expects that an element (checkbox) is checked
  - `toBeEnabled()` expects that a control is enabled
  - `toBeVisible()` expects that an element is visible
  - `toContainText()` expects that an element contains text
  - `toHaveAttribute()` expects that an element has an attribute
  - `toHaveCount()` expects that a list of elements has a given length
  - `toHaveText()` expects that an element matches text
  - `toHaveValue()` expects that an input element has a value
  - `toHaveTitle()` expects that a page has a title
  - `toHaveURL()` expects that a page has a URL

Non-Retrying Assertions – Generic Matchers

- Generic matchers do not use the await keyword as they perform immediate synchronous checks on already available values
- Examples:
  - `toBe()` expects that a value is the same
  - `toEqual()` expects that a value is similar – deep equality and pattern matching
  - `toBeCloseTo()` expects that a number is approximately equal
  - `toBeFalsy()` expects that a value is falsy, e.g., false, 0, null, etc.
  - `toBeTruthy()` expects that a value is truthy, i.e., not false, 0, null, etc.
  - `toBeNaN()` expects that a value is NaN
  - `toBeNull()` expects that a value is null

Soft Assertions

- By default, failed assertion will terminate test execution
- Failed soft assertions do not terminate test execution, but mark the test as failed

```
// Make a few checks that will not stop the test when failed...
await expect.soft(page.getByTestId('status')).toHaveText('Success');
await expect.soft(page.getByTestId('eta')).toHaveText('1 day');

// ... and continue the test to check more things.
await page.getByRole('link', { name: 'next page' }).click();
await expect.soft(page.getByRole('heading', { name: 'Make another order' }))
  .toBeVisible();
```

Configuring Assertions and Negating Matchers

- Specify the timeout:

```
// Always timeout after 10 seconds
const slowExpect = expect.configure({ timeout: 10000 });
await slowExpect(locator).toHaveText('Submit');
```

- Make an assertion soft per default:

```
// Always do soft assertions.
const softExpect = expect.configure({ soft: true });
await softExpect(locator).toHaveText('Submit');
```

Configuring Assertions and Negating Matchers

- Specify a custom expect message:

```
await expect(page.getByText('Name'), 'should be logged in')
  .toBeVisible();
```

- Matchers can be negated with `.not`:

```
expect(value).not.toEqual(0);
await expect(locator).not.toContainText('some text');
```

Asynchronous Polling

- Convert any expect to an asynchronous polling one:

```
await expect.poll(async () => {
  const response = await page.request.get('https://api.example.com');
  return response.status();
}, {
  // Custom expect message for reporting, optional.
  message: 'make sure API eventually succeeds',
  // Poll for 10 seconds; defaults to 5 seconds. Pass 0 to disable timeout.
  timeout: 10000,
}).toBe(200);
```

generic matcher

Asynchronous Polling

- Polling intervals are also supported:

```
await expect.poll(async () => {
  const response = await page.request.get('https://api.example.com');
  return response.status();
}, {
  // Probe, wait 1s, probe, wait 2s, probe, wait 10s, probe, wait 10s, probe
  // ... Defaults to [100, 250, 500, 1000].
  intervals: [1_000, 2_000, 10_000],
  timeout: 60_000
}).toBe(200);
```

Retrying Assertions

- Retry blocks of code until they are passing successfully:
  - Note that by default, `toPass()` has timeout 0 and does not respect custom expect timeout

```
await expect(async () => {
  const response = await page.request.get('https://api.example.com');
  expect(response.status()).toBe(200);
}).toPass({
  // Probe, wait 1s, probe, wait 2s, probe, wait 10s, probe, wait 10s, probe
  // ... Defaults to [100, 250, 500, 1000].
  intervals: [1_000, 2_000, 10_000],
  timeout: 60_000
});
```

Playwright `expect()` vs. Jest `expect()`

- Do not confuse Playwright's `expect()` with Jest's `expect()` library
- The latter is not fully integrated with the Playwright test runner
- Make sure to use Playwright's `expect()`

Events

- Playwright allows listening to various types of events happening on the web page, e.g.:
  - Network requests
  - Creation of child pages
  - Dedicated workers
- Several ways to subscribe to such events by
  - Waiting for events
  - Adding or removing event listeners

Waiting for Events

- Scripts might need to wait for a particular event to happen, e.g.
  - Wait for a request with the specified URL using `page.waitForRequest()`:

```
// Start waiting for request before goto. Note no await.
const requestPromise = page.waitForRequest('**/*logo*.png');
await page.goto('https://wikipedia.org');
const request = await requestPromise;
console.log(request.url());
```

  - Wait for popup window:

```
// Start waiting for popup before clicking. Note no await.
const popupPromise = page.waitForEvent('popup');
await page.getByText('open the popup').click();
const popup = await popupPromise;
await popup.goto('https://wikipedia.org');
```

Adding/Removing Event Listeners

- Sometimes, events happen in random time and instead of waiting for them, they need to be handled
  - Playwright supports traditional language mechanisms for subscribing and unsubscribing from the events:

```
page.on('request', request => console.log(`Request sent: ${request.url()}`));
const listener = request => console.log(`Request finished: ${request.url()}`);
page.on('requestfinished', listener);
await page.goto('https://wikipedia.org');

page.off('requestfinished', listener);
await page.goto('https://www.openstreetmap.org/');
```

Adding One-Off Listeners

- If a certain event needs to be handled once, there is a convenience API for that:

```
page.once('dialog', dialog => dialog.accept('2021'));
await page.evaluate("prompt('Enter a number:')");
```

Test Isolation

- Tests use the concept of test fixtures
  - There is a built-in page fixture, which is passed into a test
  - Pages are isolated between tests due to the Browser Context
    - Equivalent to a brand-new browser profile
    - Every test gets a fresh environment, even when multiple tests run in a single browser

```
import { test } from '@playwright/test';

test('example test', async ({ page, context }) => {
  // "context" is an isolated BrowserContext, created for this specific test.
  // "page" belongs to this context.
});

test('another test', async ({ page, context }) => {
  // "context" and "page" in this second test are completely
  // isolated from the first test.
});
```

Test Isolation

- Playwright isolates test from a browser’s perspective
  - If tests access common resources (e.g., a common backend), parallel tests can fail (due to race conditions)
  - Other strategies can help to achieve further isolation
    - Container Setups

Test Hooks

- Allow to group tests and add setup/teardown logic
  - `test.describe()` declares a group of tests
  - `test.beforeEach()` and `test.afterEach()` are executed before/after each test
  - `test.beforeAll()` and `test.afterAll()` are executed once per worker before/after all tests

```
import { test, expect } from '@playwright/test';

test.describe('navigation', () => {
  test.beforeEach(async ({ page }) => {
    // Go to the starting url before each test.
    await page.goto('https://playwright.dev/');
  });

  test('main navigation', async ({ page }) => {
    // Assertions use the expect API.
    await expect(page)
      .toHaveURL('https://playwright.dev/');
  });
});
```

Test Hooks

- Code can also be executed once before running all tests via
  - Project dependencies
  - `globalSetup`/`globalTeardown`
    - Lacks some features!

`tests/global.setup.ts`:

```ts
import { test as setup } from '@playwright/test';

setup('create new database', async ({ }) => {
  console.log('creating new database...');
  // Initialize the database
});
```

`playwright.config.ts`:

```ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  // ...
  projects: [
    {
      name: 'setup db',
      testMatch: /global\.setup\.ts/,
    },
    {
      name: 'chromium with db',
      use: { ...devices['Desktop Chrome'] },
      dependencies: ['setup db'],
    },
  ]
});

// ... or ...

import { defineConfig } from '@playwright/test';

export default defineConfig({
  globalSetup: require.resolve('./global-setup'),
  globalTeardown: require.resolve('./global-teardown'),
});
```

Timeouts

- Multiple configurable timeouts for various tasks:

Timeout	Default	Description

Test timeout	30_000 ms	Timeout for each test
		SET IN CONFIG
		`{ timeout: 60_000 }`
		OVERRIDE IN TEST
		`test.setTimeout(120_000)`

Expect timeout	5_000 ms	Timeout for each assertion
		SET IN CONFIG
		`{ expect: { timeout: 10_000 } }`
		OVERRIDE IN TEST
		`expect(locator).toBeVisible({ timeout: 10_000 })`

Global timeout	0 ms	Maximum time in milliseconds the whole test suite can run **SET IN CONFIG** `{ globalTimeout: 3_600_000 }`

Test Timeout

- Playwright enforces a timeout for each test
  - 30 seconds by default
  - Test timeout includes
    - Time spent by the test function
    - Fixture setups
    - `beforeEach()` hooks
  - Additional separate timeout, of the same value, is considered after the test function has finished, and includes
    - Fixture teardowns
    - `afterEach()` hooks
  - The same timeout value also applies to `beforeAll()` and `afterAll()` hooks
    - These do not share time with any test

Test Timeout

- Can be set in the config …

```ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 120_000,
});
```

- … or for a single test …

```ts
import { test, expect } from '@playwright/test';

test('slow test', async ({ page }) => {
  test.slow(); // Easy way to triple the default timeout
  // ...
});

test('very slow test', async ({ page }) => {
  test.setTimeout(120_000);
  // ...
});
```

Test Timeout

- Can also be set in test hooks:

```
import { test, expect } from '@playwright/test';

test.beforeAll(async () => {
  // Set timeout for this hook.
  test.setTimeout(60_000);
});

test.beforeEach(async ({ page }, testInfo) => {
  // Extend timeout for all tests running this hook by 30 seconds.
  testInfo.setTimeout(testInfo.timeout + 30_000);
});
```

Except Timeout

- Auto-retrying assertions have a separate timeout
  - 5 seconds by default
  - Assertion timeout is unrelated to the test timeout

- Can be set in the config …

```ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    timeout: 10_000,
  },
});
```

- … or for a single test …

```ts
import { test, expect } from '@playwright/test';

test('example', async ({ page }) => {
  await expect(locator)
    .toHaveText('hello', { timeout: 10_000 });
});
```

Retries

- Playwright offers a way to automatically re-run a test when it fails
  - Useful when a test is flaky and fails intermittently
  - Test retries are configured in the configuration file

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  // Give failing tests 3 retry attempts
  retries: 3,
});
```

- Tests will be categorized as follows:
  - passed: tests that passed on the first run
  - flaky: tests that failed on the first run, but passed when retried
  - failed: tests that failed on the first run and failed all retries

Parallelism

- Playwright runs tests in parallel
- Uses several worker processes to execute tests
  - By default
    - Test files are run in parallel
    - Tests in a single file are run in order, in the same worker process
  - The `TestInfo` class provides information on the executing `workerIndex`:

```
test.info().workerIndex
```

Parallelism

- `test.describe.configure()` can be used to configure tests in a single file to be run in parallel:

```
import { test } from '@playwright/test';
test.describe.configure({ mode: 'parallel' });
test('runs in parallel 1', async ({ page }) => { /* ... */ });
test('runs in parallel 2', async ({ page }) => { /* ... */ });
```

- Alternatively, to execute all tests in this fully-parallel mode, update the configuration file:

```
import { defineConfig } from
'@playwright/test';
export default defineConfig({
  fullyParallel: true,
});
```

- Fully-parallel mode can also be activated per project

Parallelism

- Opt specific tests out of fully-parallel mode:

```
test.describe('runs in parallel with other describes', () => {
  test.describe.configure({ mode: 'default' });
  test('in order 1', async ({ page }) => {});
  test('in order 2', async ({ page }) => {});
});
```

Serial Mode

- Inter-dependent tests can be annotated as serial
  - If one of the serial tests fails, all subsequent tests are skipped
  - All tests in a group are retried together
- Caution: Playwright creates an isolated Page object for each test
  - Inter-dependent tests probably want to reuse a single Page instance, as shown on the right

```
import { test, type Page } from '@playwright/test';

// Annotate entire file as serial.
test.describe.configure({ mode: 'serial' });

let page: Page;

test.beforeAll(async ({ browser }) => {
  page = await browser.newPage();
});

test.afterAll(async () => {
  await page.close();
});

test('runs first', async () => {
  await page.goto('https://playwright.dev/');
});

test('runs second', async () => {
  await page.getByText('Get Started').click();
});
```

Page Object Models

- Approach to structure test suite
- Page Object Models (POMs) represent a part of the web application
  - E.g., a home page, a listings page, a checkout page
- POMs simplify test authoring by creating a higher-level API
  - Simplifies maintenance by capturing element selectors in one place
  - Create reusable code to avoid repetition

```ts
import type { Page, Locator } from '@playwright/test';

export class TodoPage {
  private readonly inputBox: Locator;
  private readonly todoItems: Locator;

  constructor(public readonly page: Page) {
    this.inputBox = this.page.locator('input.new-todo');
    this.todoItems = this.page.getByTestId('todo-item');
  }

  async goto() {
    await this.page.goto('https://demo.playwright.dev/todomvc/');
  }
  async addToDo(text: string) {
    await this.inputBox.fill(text);
    await this.inputBox.press('Enter');
  }
  async remove(text: string) {
    const todo = this.todoItems.filter({ hasText: text });
    await todo.hover();
    await todo.getByLabel('Delete').click();
  }
  async removeAll() {
    while ((await this.todoItems.count()) > 0) {
      await this.todoItems.first().hover();
      await this.todoItems.getByLabel('Delete').first().click();
    }
  }
}
```

```ts
const { test } = require('@playwright/test');
const { TodoPage } = require('./todo-page');

test.describe('todo tests', () => {
  let todoPage;

  test.beforeEach(async ({ page }) => {
    todoPage = new TodoPage(page);
    await todoPage.goto();
    await todoPage.addToDo('item1');
    await todoPage.addToDo('item2');
  });

  test.afterEach(async () => {
    await todoPage.removeAll();
  });

  test('should add an item', async () => {
    await todoPage.addToDo('my item');
    // ...
  });

  test('should remove an item', async () => {
    await todoPage.remove('item1');
    // ...
  });
});
```

Fixtures

- Test fixtures are used to establish the environment for each test
  - They give the test everything it needs and nothing else
  - Fixtures are isolated between tests
  - With fixtures, one can group tests based on their meaning, instead of their common setup
- One built-in fixture has already been shown: the `page` fixture

```
import { test, expect } from '@playwright/test';

test('basic test', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  await expect(page).toHaveTitle(/Playwright/);
});
```

- Other fixtures include
  - `context: BrowserContext`
  - `browser: Browser`
  - `browserName: string`
  - `request: APIRequestContext`

Fixtures vs. `before`/`after` Hooks

- Fixtures encapsulate setup and teardown in the same place
  - So if you have an after hook that tears down what was created in a before hook, consider turning them into a fixture
- Fixtures are reusable between test files
  - You can define them once and use them in all your tests
  - So if you have a helper function that is used in multiple tests, consider turning it into a fixture
- Fixtures are on-demand
  - You can define as many fixtures as you'd like
  - Playwright will setup only the ones needed by your test and nothing else

Fixtures vs. `before`/`after` Hooks

- Fixtures are composable
  - They can depend on each other to provide complex behaviors
- Fixtures are flexible
  - Tests can use any combination of fixtures to precisely tailor the environment to their needs, without affecting other tests
- Fixtures simplify grouping
  - You no longer need to wrap tests in describes that set up their environment, and are free to group your tests by their meaning instead

Include Custom Fixtures

- `test.extend()` can be used to create a new test object that will include a new custom fixture
- A fixture can be used by specifying it as test function argument
  - Fixtures are also available in hooks and other fixtures

```
import { test as base } from '@playwright/test';
import { TodoPage } from './todo-page';

// Extend basic test by providing a "todoPage" fixture.
const test = base.extend<{ todoPage: TodoPage }>({
  todoPage: async ({ page }, use) => {
    // Set up the fixture.
    const todoPage = new TodoPage(page);
    await todoPage.goto();
    await todoPage.addToDo('item1');
    await todoPage.addToDo('item2');

    // Use the fixture value in the test.
    await use(todoPage);

    // Clean up the fixture
    await todoPage.removeAll();
  },
});

test('should add an item', async ({ todoPage }) => {
  await todoPage.addToDo('my item');
  // ...
});
test('should remove an item', async ({ todoPage }) => {
  await todoPage.remove('item1');
  // ...
});
```

Overriding Fixtures

- Existing fixtures can be overridden:
  - The following example overrides the page fixture so that it automatically navigates to https://demo.playwright.dev/todomvc/

```
import { test as base } from '@playwright/test';

export const test = base.extend({
  page: async ({ page }, use) => {
    await page.goto('https://demo.playwright.dev/todomvc/');
    await use(page);
  },
});
```